The first homework gives you a first-hand experience on how to implement MapReduce computations in Spark and to run them on a cluster platform, made available to the course through the university cloud infrastructure (CloudVeneto). Specifically, the homework requires you to implement and test a 2-round coreset-based strategy for k-center clustering with an additional *fairness constraint* imposed on the returned centers.

ACCESS TO CLOUD VENETO. In order to access and use the CloudVeneto cluster, you must strictly follow the rules indicated in the User guide for the CloudVeneto cluster which you can find in the same section as this page.

FAIR K-CENTER. Let U be a set of points in $\mathbb{R}^D$, where each point has an extra label $g_p \in \{A, B\}$ indicating a demographic group (e.g., female or male). We call one such set U as a **2-labeled pointset**, and define U_A, U_B the subsets of U corresponding to the two groups.

The *fair k-center problem* is defined as follow

- **Input:** a 2-labeled pointset U and 2 positive integers k_A and k_B, with $k_A \leq |U_A|$ and $k_B \leq |U_B|$.
- **Output:** a set $S \subseteq U$ of $k = k_A + k_B$ centers, exactly k_A from U_A and k_B from U_B, which minimize $\max_{x \in U} \mathrm{dist}(x, S)$.

For the homework, **dist** will be the ***standard Euclidean distance function***.

ALGORITHMS. In the homework you must implement the following approximation algorithms for fair k-center.

- **Sequential algorithm: Fair-FFT.** A simple variant of Farthest-First Traversal where $k = k_A + k_B$ centers are selected keeping track of their demographic groups. As soon as the current set of centers has exhausted the budget for a group (i.e., has k_A centers from U_A or k_B centers from U_B), the remaining iterations will pick centers only from the other group.
- **MapReduce algorithm: MR-Fair-FFT.** A simple variant of the 2-round MR-Farthest-First Traversal seen in class, where in both rounds Fair-FFT is used instead of the standard Farthest-First Traversal. In Round 1, you are allowed to select more than $k_A + k_B$ points from each partition, thus yielding a larger coreset, if this provides better accuracy without excessive time penalty.

REPRESENTATION OF POINTS

(JAVA Users) Points must be represented as instances of the class Vector of package mllib.linalg and can be manipulated through static methods offered by the class Vectors in the same package. For example, method Vectors.dense(x) transforms an array x of double into an instance of class Vector, while method Vectors.sqdist(x,y) computes the squared Euclidean distance between two instances, x and y, of class Vector. Details on these classes can be found in the Spark Java API. **WARNING.** Make sure to use the classes from the org.apache.spark.mllib package. There are classes with the same name in org.apache.spark.ml package which are functionally equivalent, but incompatible with those of the org.apache.spark.mllib package.

(PYTHON Users) Points must be represented as tuples of float (i.e., point = (x1, x2, ...)). Although Spark provides the class Vector also for Python (see pyspark.mllib package), its performance is very poor and its more convenient to use tuples, especially for points from low-dimensional spaces.

INPUT FORMAT. Your code must assume that the input set U is from $\mathbb{R}^D$ (for some arbitrary D), and it is given in input as a file, where each row contains one point stored with the coordinates (reals) separated by comma (',') and, at then end, a character, A or B, representing its group label. For instance, if $D = 3$, a point p = (1.5,6.0,2.3) of group U_A, will occur in the file as line: 1.5,6.0,2.3,A.

ASSIGNMENT for HW1. You must do the following tasks.

1) Write a method/function FairFFT which implements the Fair-FFT algorithm described above. Both the input set U and the solution S must be represented as a lists (ArrayList in Java) of pairs (p, g_p), where $p \in \mathbb{R}^D$ is a point and $g_p \in \{A, B\}$ is its group label.

2) Write a method/function MRFairFFT which implements the MR-Fair-FFT algorithm described above. The input set U must be represented as a RDD of pairs (p, g_p), where $p \in \mathbb{R}^D$ is a point and $g_p \in \{A, B\}$ is its group label, while the solution S must be represented as a ArrayList/list of such pairs. As partitions for Round 1, use the Spark partitions of the set U, which can be accessed through the mapPartitionsToPair (Java) or mapPartitions (Python) methods.

?

WARNING: the set U or any data structure of similar size must be stored exclusively in distributed form, using RDDs. Otherwise, the evaluation will be penalized.

3) Write a program GxxHW1.java (for Java users) or GxxHW1.py (for Python users), where xx is your 2-digit group number (e.g., 04 or 25), which receives in input, as command-line arguments, a path to the file storing the input points, and 3 integers k_A, k_B, L, and does the following:

- Prints the command-line arguments.
- Reads the set U of input points into an RDD -called **inputPoints**-, subdivided into L partitions.
- Prints: $N = |U|, N_A = |U_A|$ and $N_B = |U_B|$.
- Runs MRFairFFT to computes a solution S to the fair k-center problem for the instance (U, k_A, k_B).
- Prints the centers of S together with their group label (on center per line) and the value of the objective function $\max_{x \in U} \mathrm{dist}(x, S)$
- Print the time required by the execution of MRFairFFT in ms.

The program must be executable from command prompt in the same way as the WordCountExample program that we provided.

OUTPUT FORMAT: In the same section as this page you will find some datasets and outputs corresponding to specific input configurations using these datasets. Your program must *STRICTLY ADHERE TO THE OUTPUT FORMAT* used in **these examples (to be added)**. Any deviation from this format will be penalized.

4) Test your program as follows:

- Test and debug the program in local mode on your PC to make sure that it runs correctly.
- Only after you are confident that your program runs correctly, run it on the cluster using the datasets which have been preloaded in the HDFS, and fill the table given in **this word form (to be added)** with the results of the specified experiments.
- **To avoid congestion, groups with even (resp., odd) group number must use the clusters in even (resp., odd) days. Also:**
  - Do not run several instances of your program at once
  - Do not use more than 16 executors
  - Try your program on a smaller dataset first.
  - Remember that if your program is stuck for more than 1 hour, its execution will be automatically stopped by the system.

USE OF GENERATIVE AI TOOLS. The use of such tools is not forbidden, but must be explicitly declared in the word form to be returned with the code, as indicated above. In all cases, each student must be fully aware of the code submitted by his/her group, and is responsible for the code's correctness and efficiency. We recall that the questions about the homeworks can be asked in the exams.

SUBMISSION INSTRUCTIONS. Each group must **submit a zipped folder GxxHW1.zip, where xx is your group number. The folder must contain the program (GxxHW1.java or GxxHW1.py) and a pdf file GxxHW1form.pdf** (suitably renaming xx with your group number), which is the pdf version of the word document with the aforementioned table. Only one student per group must do the submission using the link provided in the Homework 1 section. Make sure that your code is free from compiling/run-time errors and that you comply with the specification, otherwise your grade will be penalized.

If you have questions about the assignment, contact the teaching assistants (TAs) by email to bdc-course@dei.unipd.it . The subject of the email must be "HW1 - Group xx", where xx is your group ID. If needed, a zoom meeting between the TAs and the group will be organized.

Last modified: Thursday, 2 April 2026, 9:07 PM